

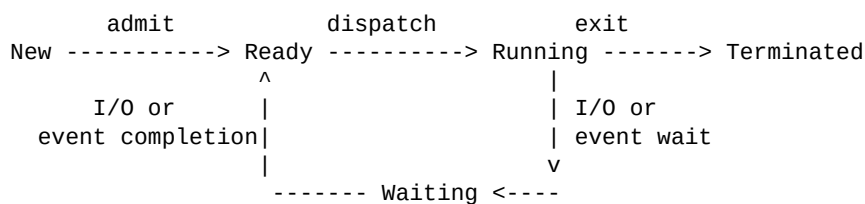
Operating System – Unit 2: Process Management

Notes by Debkanta Maity – Debkanta Study Zone

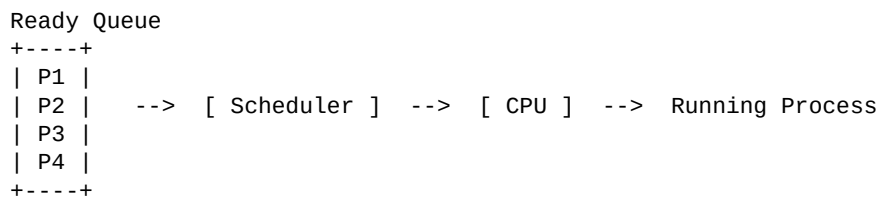
Topics covered:

- Process Concept
- Process States
- Process Control Block (PCB)
- Process Scheduling
- Threads
- Inter-Process Communication
- CPU Scheduling
- Context Switching

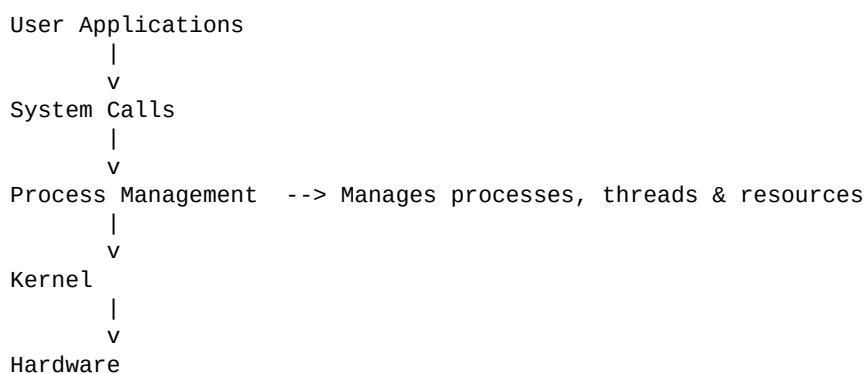
Process State Diagram (cover page):



CPU Scheduling – simple view (cover page):



Layered view of process management (cover page):



Key concepts (cover page):

- Process
 - Thread
 - PCB (Process Control Block)
 - Ready Queue
 - Scheduler
 - Dispatcher
 - Context Switch
-

Unit - 2: Operating Systems

SYLLABUS: Processes: Definition, Process Relationship, Different states of a Process, Process State transitions, Process Control Block (PCB), Context switching. Thread: Definition, Various states, Benefits of threads, Types of threads, Concept of multithreads. Process Scheduling: Foundation and Scheduling objectives, Types of Schedulers, Scheduling criteria: CPU utilization, Throughput, Turnaround Time, Waiting Time, Response Time; Scheduling algorithms: Pre-emptive, Non pre-emptive, FCFS, SJF, RR; Multiprocessor scheduling: Real Time scheduling: RM and EDF.

PROCESSES

Definition:

A process is the central concept in an operating system (OS) and is defined as a program in execution. Unlike a passive program file stored on a disk, a process is an active entity that requires and uses system resources, such as the CPU, memory, and I/O devices, to accomplish a task.

Process Relationships:

Processes often interact with each other to share data, cooperate on a task, or simply manage the system structure. The relationships are categorized primarily into two groups:

1. Structure / Hierarchy Relationships

Parent and Child Processes:

- A Parent process creates a new process (the Child process) using a system call (like fork() in Unix/Linux).
- This establishes a hierarchical relationship (a process tree). The child process inherits some resources and properties from the parent.

Sibling Processes:

- Processes that share the same parent process. They execute independently but may still communicate with each other.

2. Interaction Relationships

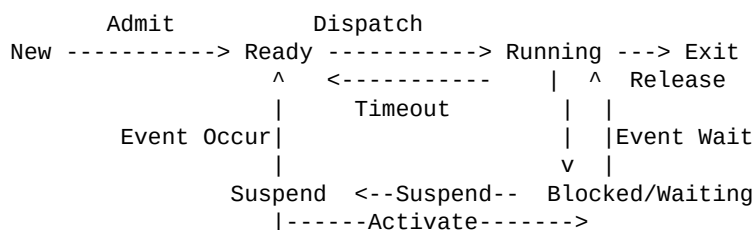
Independent Processes:

- Processes that do not share data or resources with other processes.
- Their execution does not affect the execution or state of any other process. (e.g., running a media player and a web browser simultaneously).

Cooperating Processes:

- Processes that can affect the execution of other processes because they share data or resources.
- They typically communicate using Inter-Process Communication (IPC) mechanisms like pipes, shared memory, or message passing to work together (e.g., a compiler and a linker).

Process State Transitions (overview diagram):



Seven Process States Explained

1. NEW: The process is being created and hasn't been admitted to the pool of executable processes yet.

The transitions show how the OS manages both CPU usage and memory resources:

I. Execution (CPU) Transitions (The Inner Loop)

These transitions occur in RAM:

- **Ready --Dispatch--> Running:** The Dispatcher (a short-term scheduler) selects the process to execute and allocates the CPU to it.
- **Running --Timeout--> Ready:** The process's allocated time slice expires, and the Preempt mechanism returns it to the Ready Queue to wait for its next turn.
- **Running --Event Wait--> Blocked/Waiting:** The process executes an operation that requires waiting for an event (like an I/O request), so it yields the CPU.
- **Blocked/Waiting --Event Occur--> Ready:** The event the process was waiting for has completed (e.g., the I/O operation finished), making it eligible for the CPU again.
- **Running --Release--> Exit:** The process finishes its execution or is terminated by the OS.

These transitions involve moving the process between RAM and disk, usually to free up space in RAM:

- **Blocked/Waiting --Suspend--> Suspend:** The OS decides to move a process that is already waiting for an event out of main memory and onto the disk. This frees up RAM for other processes.
- **Suspend --Activate--> Blocked/Waiting:** The OS moves the suspended process back into main memory to prepare it for potential movement to the Ready state once its waiting event occurs.
- **Ready --Suspend--> Suspend:** (Implicitly, the diagram shows a direct Time Slice/Event Wait/Preempt to Suspend, but this is typically interpreted as a Ready -> Suspend transition). This means a Ready process (which isn't waiting for anything) is swapped out to disk, often when the OS needs to manage a memory shortage for high-priority processes.
- **Suspend --Event Occur--> Ready:** The event the suspended process was waiting for occurs. The process is then brought back into memory and placed in the Ready Queue, as it is now ready to run.

A Process Control Block (PCB) is a crucial data structure used by the Operating System (OS) to manage and control processes. It contains all the information required by the OS to keep track of a process at any moment. Whenever a process is created, the OS generates a PCB for it, and when the process terminates, the PCB is destroyed. The PCB acts as the complete record or profile of a process, and supports functions like process scheduling, context switching, and resource allocation.

```

+-----+
|      Pointer      |

```

Process State
Process Number
Process Counter
Registers
Memory Limit
List of open Files
:

Structure of the Process Control Block

A Process Control Block (PCB) is a data structure used by the operating system to manage information about a process. The process control keeps track of many important pieces of information needed to manage processes efficiently. The diagram helps explain some of these key data items.

Pointer:

It is a stack pointer that is required to be saved when the process is switched from one state to another to retain the current position of the process.

Process state:

It stores the respective state of the process.

Process number:

Every process is assigned a unique id known as process ID or PID which stores the process identifier.

Program counter:

Program counter stores the counter, which contains the address of the next instruction that is to be executed for the process.

Register:

Registers in the PCB, it is a data structure. When a process is running and its time slice expires, the current value of process-specific registers would be stored in the PCB and the process would be swapped out. When the process is scheduled to be run, the register values are read from the PCB and written to the CPU registers. This is the main purpose of the registers in the PCB.

Memory Limits:

This field contains the information about memory management system used by the operating system. This may include page tables, segment tables etc.

List of Open files:

This information includes the list of files opened for a process.

Functions / Role of PCB:

- Supports Context Switching:** When the CPU switches from one process to another, the OS saves the current process's state in its PCB and loads the next process's PCB. This allows smooth multitasking.
 - Helps in Process Scheduling:** Scheduler reads PCB fields like priority, process state, and queue pointers to decide which process should run next.
 - Ensures Process Management:** PCB records everything needed for creation, execution, suspension, and termination of a process.
 - Enables Memory and Resource Protection:** Memory and resource information in PCB ensures processes do not interfere with each other.
 - Maintains Execution Information:** The PCB acts as the repository for all process-related execution details.
-

Context Switching (Thread)

Definition: Context switching is the process of saving the state of a running thread and restoring the state of another thread so that the CPU can continue execution from the exact point where each thread left off.

In thread context switching:

- The OS saves register values, program counter, stack pointer, and thread-specific data of the current thread.
 - Then it loads the saved state of another thread.
-

Thread

A thread is the smallest sequence of programmed instructions that can be managed independently by the operating system. It is also called a lightweight process because multiple threads can exist within a single process and share its resources.

Threads within the same process share:

- Code segment
- Data segment
- Open files
- Memory address space

Various States of a Thread

Threads generally pass through the following states:

1. **New:** Thread is created but not yet started.
2. **Ready:** Thread is ready to run and waiting for CPU time.
3. **Running:** Thread is currently executing on the CPU.
4. **Blocked/Waiting:** Thread is waiting for a resource, I/O operation, lock, or event.
5. **Timed Waiting:** Thread is waiting for a specified time (e.g., sleep, timeout).
6. **Terminated:** Thread has finished execution or stopped due to an error.

Benefits of Thread:

1. **Faster Context Switching:** Threads share the same memory space, so switching between them is very fast.
2. **Efficient Communication:** Threads of a process can share data easily without using complex inter-process communication (IPC).
3. **Better Resource Utilization:** Threads share code, memory, and files, reducing overhead.
4. **Improved Application Performance:** Multithreading allows parallelism, speeding up processing on multi-core CPUs.
5. **Responsiveness:** Threads keep applications responsive. Example: UI thread + background worker thread.

Types of Threads:

Threads are classified into User-Level Threads, Kernel-Level Threads, and Thread Mapping Models (Many-to-One, One-to-One, Many-to-Many).

1. User-Level Threads (ULT)

Definition: User-level threads are threads managed entirely in user space with the help of a thread library. The operating system is not aware of these threads.

Working:

- All thread operations (creation, scheduling, synchronization) are done by a user-level thread library.

- Only one kernel thread represents the entire process.

Features:

- Very fast thread creation and context switching.
- Portable across different operating systems.

Advantages:

- No kernel intervention -> switching is fast.
- Can run even on operating systems that do not support threading.
- Lower resource consumption.

Disadvantages:

- If one thread performs a blocking system call, the entire process blocks.
- Cannot use multiple processors simultaneously (no true parallelism).

Examples:

- Java Green Threads
- POSIX ULT libraries
- GNU Portable Threads

2. Kernel-Level Threads (KLT)

Definition: Kernel-level threads are managed directly by the OS kernel. The kernel knows and schedules every thread individually.

Working:

- Each user thread corresponds to a kernel thread.
- Kernel performs scheduling and context switching.

Features:

- True parallelism on multi-core systems.
- Each thread has its own kernel data structure.

Advantages:

- If one thread blocks on I/O, others continue running.
- Better CPU utilization.
- Supports multi-core execution.

Disadvantages:

- Thread creation and switching are slower due to kernel overhead.
- Consumes more memory.

Examples:

- Linux kernel threads
- Windows threads
- Solaris kernel threads

3. Many-to-One Thread Model

Definition: Multiple user-level threads are mapped to one kernel thread.

Working:

- User-level library manages threads.
- Kernel sees only one single thread.

Advantages:

- Very fast switching.
- Easy to implement.

Disadvantages:

- If one thread blocks, the entire process blocks.
- No parallelism (only one kernel thread exists).

Used in:

- Early Java Green Thread implementations.
- Some early UNIX thread libraries.

4. One-to-One Thread Model

Definition: Each user-level thread is mapped to one kernel thread.

Working:

- OS manages each thread directly.
- Each ULT corresponds to a KLT.

Advantages:

- True parallel execution on multiprocessor systems.
- Blocking one thread does not block others.

Disadvantages:

- Higher overhead due to many kernel threads.
- Too many threads can overload the system.

Used in:

- Windows threading model.
- Linux pthreads.

5. Many-to-Many Thread Model

Definition: Many user-level threads are mapped to a smaller or equal number of kernel threads.

Working:

- User-level threads are created in user space.
- Kernel threads execute them based on availability.

Advantages:

- Combines benefits of both ULT and KLT.
- True parallelism possible.
- Blocking a thread does not block all.
- Highly scalable and flexible.

Disadvantages:

- More complex to implement.

Used in:

- Solaris 9
- Modern Windows Thread Pool
- Some POSIX thread libraries.

Short Note on Multithreading

Multithreading is a concept in operating systems where a single process is divided into multiple smaller units of execution called threads.

Each thread runs independently but shares the same process resources, such as memory, files, and code.

Key idea:

- A process can have multiple threads, each performing a different task.
- All threads run within the same address space.
- Threads are lightweight and require fewer resources than full processes.

Features of Multithreading:

1. Multiple tasks within the same program can run concurrently.
2. Threads share code and data, reducing memory usage.
3. Thread creation and switching require less overhead than processes.
4. Improves system responsiveness and CPU utilization.

Types of Multithreading:

1. **User-Level Multithreading** – Controlled by user libraries.
2. **Kernel-Level Multithreading** – Controlled by the operating system.
3. **Hybrid Multithreading** – Combination of both.

Example:

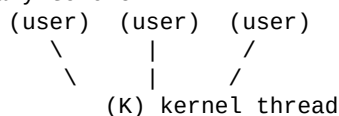
Consider a web browser (one process):

- One thread loads images.
- Another thread handles user input.
- Another thread loads the webpage data.

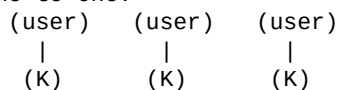
All work together, making the browser fast and responsive.

Multithreading Models (diagram):

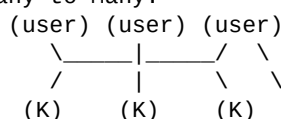
Many-to-One:



One-to-One:



Many-to-Many:



Process Scheduling

Foundation of Process Scheduling

Process scheduling is the core function of the Operating System that selects which process/thread should run on the CPU at a given time.

Its main goal is to maximize CPU utilization, reduce waiting time, and ensure fairness among processes.

Key Points in the Foundation:

- A system has more processes than CPUs, so scheduling is required.
- Scheduling decides when a process enters ready, running or waiting states.
- Schedulers use algorithms to decide the execution order (FIFO, Round Robin, Priority etc).
- The OS performs context switching to save and load process states.

Scheduling Objects:

Scheduling objects are the entities that the scheduler manages. These include:

Process Control Block (PCB):

Contains process information like PID, state, registers, priority, etc.

Ready Queue:

Processes waiting for CPU time are stored in this queue.

CPU Scheduling Parameters:

- Priority
- CPU burst time
- Arrival time
- Deadline

State Information:

Process states: new, ready, running, waiting, terminated.

Scheduling Algorithms – objects such as:

- Time quantum (in Round Robin)
- Burst time estimators
- Priority levels

Threads:

If the OS supports multithreading, the scheduler may schedule threads instead of processes.

Types of Schedulers:

1. Long-Term Scheduler (Job Scheduler)

Definition: The Long-Term Scheduler decides which jobs (processes) should be admitted into the system for processing. It controls the degree of multiprogramming, i.e., how many processes are in memory at once.

Functions:

- Selects processes from the job pool on secondary storage.
- Moves selected processes into main memory and places them in the ready queue.
- Balances between I/O-bound and CPU-bound processes (to achieve high CPU utilization).

Characteristics:

- Runs least frequently among the three schedulers.
- Used mostly in batch processing systems.

Example:

If the system becomes overloaded, the long-term scheduler may stop admitting new jobs.

Advantages:

- Controls system load.
- Helps improve system throughput.

2. Short-Term Scheduler (CPU Scheduler)

Definition: The Short-Term Scheduler decides which ready process in the ready queue should be assigned the CPU next.

Functions:

- Performs context switching.
- Implements CPU scheduling algorithms (FCFS, SJF, Priority, RR, etc.)
- Selects the next process to run every few milliseconds.

Characteristics:

- Runs very frequently (1000s of times per second).

- Most critical scheduler for system performance.
- Decides the execution order of processes.

Example:

Choosing between processes with different priorities or time quantum values.

Advantages:

- Ensures fair CPU allocation.
- Increases CPU utilization.
- Reduces waiting and response time.

3. Mid-Term Scheduler (Swapper)

Definition: The Mid-Term Scheduler temporarily removes processes from memory (swap out) and later brings them back (swap in). It is used for memory management and controlling multiprogramming level.

Functions:

- Suspends processes when memory is low.
- Moves suspended processes to secondary memory.
- Re-admits (swaps in) processes when enough memory becomes available.

Characteristics:

- Used mainly in time-sharing systems.
- Balances the load on the system.

Example:

If there are too many processes in memory causing thrashing, the mid-term scheduler suspends some processes.

Advantages:

- Prevents memory overload.
- Improves CPU and memory efficiency.
- Supports smooth multitasking.

CPU Scheduling Criteria

CPU scheduling is the process of selecting which process should be executed by the CPU at any given time. To evaluate and compare different CPU scheduling algorithms, several performance criteria are used. These criteria help determine how efficiently the CPU is utilized and how well the system responds to user needs.

1. CPU Utilization

Definition: The percentage of time the CPU is actively executing processes.

Goal: Maximize CPU utilization (keep CPU as busy as possible)

Ideal Value: Ranges from 40% (light systems) to 90% (heavy loads).

Importance: Higher utilization means less idle time and better performance.

2. Throughput

Definition: The number of processes completed per unit time.

Example: If 10 processes finish in 5 seconds -> throughput = 2 processes/sec.

Goal: Maximize throughput.

Importance: Higher throughput indicates more work is being done.

3. Turnaround Time

Definition: Total time taken from process submission to process completion.

Formula: Turnaround Time = Completion Time – Arrival Time

Includes waiting time, execution time, I/O time.

Goal: Minimize turnaround time for better user experience.

4. Waiting Time

Definition: Total time a process spends in the ready queue waiting for CPU.

Formula: Waiting Time = Turnaround Time – Burst Time

Goal: Minimize waiting time.

Importance: Lower waiting time improves system responsiveness.

5. Response Time

Definition: Time from process submission until the first response is produced.

In interactive systems (e.g., GUI, servers), response time is critical.

Goal: Minimize response time.

Importance: Faster response improves user satisfaction.

Pre-emptive vs Non-Pre-emptive Scheduling

Factor	Pre-emptive Scheduling	Non-Pre-emptive Scheduling
Control	OS has full control	Process controls CPU until completion.
CPU Interruption	Allowed	Not Allowed
Context Switching	Occurs frequently	Rare or no switching
Response Time	Fast	Slower
Starvation	Possible	Less likely
Complexity	High	Low
Examples	RR, SRTF, Pre-emptive Priority	FCFS, Non-preemptive SJF.

Discuss the CPU Scheduling Algorithms

1. First-Come, First-Served (FCFS)

The First-Come, First-Served scheduling algorithm is the simplest type of CPU scheduling algorithm. In FCFS, the process that arrives first in the ready queue gets executed first. This approach resembles a queue at a grocery store, where the first customer is served first.

Characteristics of FCFS:

- **Non-Preemptive:** Once a process is allocated the CPU, it runs to completion without interruption.
- **Simplicity:** Easy to implement and understand.
- **Fairness:** All processes get a chance to be executed in the order of arrival.

Drawbacks of FCFS:

- **Convoy Effect:** Short processes can get stuck waiting behind long ones, which leads to poor average wait times.
- **Throughput:** Not optimal for time-sharing systems where response time is critical.

Example of FCFS:

Process	Arrival Time	Burst Time
P1	0	4

Process	Arrival Time	Burst Time
P2	1	3
P3	2	2

In this example, the processes would be executed in the order of P1, P2, P3, leading to a total waiting time of:

Waiting Time for P1 = 0
Waiting Time for P2 = 4 (from P1)
Waiting Time for P3 = 7 (4 from P1 + 3 from P2)

Average Waiting Time = $(0 + 4 + 7) / 3 = 3.67$

2. Shortest Job First (SJF)

Shortest Job First is a scheduling algorithm that selects the process with the smallest execution time. This non-preemptive algorithm minimizes the average waiting time for processes and can significantly improve system efficiency.

Characteristics of SJF:

- **Optimal for Average Waiting Time:** SJF minimizes the average waiting time compared to any other scheduling method.
- **Non-Preemptive:** Once a process starts executing, it runs to completion.

Drawbacks of SJF:

- **Starvation:** Longer processes may never get executed if shorter processes continue to arrive.
- **Difficulty in Estimation:** It can be challenging to know the exact burst time of processes in advance.

Example of SJF:

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	3

In this case, the order of execution would be P2, P4, P1, then P3:

Waiting Time for P2 = 0
Waiting Time for P4 = 3 (time until P2 finishes)
Waiting Time for P1 = 7 (3 + 4 for P2 and P4)
Waiting Time for P3 = 16 (7 + 9 for P1)

Average Waiting Time = $(0 + 3 + 7 + 16) / 4 = 6.5$

3. Round Robin (RR)

Round Robin is one of the fairest scheduling algorithms, designed primarily for time-sharing systems. It treats all processes equally and allocates a fixed time slice, known as a quantum, to each process.

Characteristics of RR:

- **Preemptive:** If a process doesn't finish execution within its time slice, it is moved to the back of the queue.
- **Fair Process Sharing:** All processes receive an equal opportunity to execute over time.

Drawbacks of RR:

- **Time Slice Value:** Choosing a quantum that's too high can lead to increased waiting time, while a lower quantum can lead to context-switching overhead.

- **Not Optimal for Short Processes:** Longer waiting might lead to inefficiency for systems requiring quick responses.

Example of RR (Quantum: 2):

Process	Arrival Time	Burst Time
P1	0	4
P2	1	5
P3	2	6

The execution order would be: P1, P2, P3, P1 (again), P2, P3, leading to varied waiting times for each process.

Waiting Time for P1 = 2 + 4 = 6 (after P1, P2, and P3 complete their first slice)

Waiting Time for P2 = 2 + 2 = 4 (after its first slice)

Waiting Time for P3 = 2 (after P3 finishes first slice)

Average Waiting Time = (6 + 4 + 2) / 3 = 4

4. Priority Scheduling

Priority Scheduling is another crucial algorithm where every process is assigned a priority. The CPU is allocated to the process with the highest priority (the lowest number is typically the highest priority).

Characteristics of Priority Scheduling:

- **Preemptive or Non-Preemptive:** Can be either depending on the system design.
- **Efficient for critical processes:** Very useful for real-time systems.

Drawbacks of Priority Scheduling:

- **Starvation:** Lower priority processes may wait indefinitely if higher priority processes keep coming.
- **Preemption Overhead:** Can lead to constant context switching if done poorly.

Example of Priority Scheduling:

Process	Arrival Time	Burst Time	Priority
P1	0	4	2
P2	1	2	1
P3	2	1	3

In this scenario, with P2 having the highest priority, the execution order would be P2, P1, P3:

Waiting Time for P1 = 1 (time until P2 finishes execution)

Waiting Time for P2 = 0 (starts immediately)

Waiting Time for P3 = 3 (after P1 and P2 complete)

Average Waiting Time = (1 + 0 + 3) / 3 = 1.33

Multiprocessor Scheduling

Multiprocessor scheduling refers to the method of scheduling processes on a system that has more than one processor (CPU). Its goal is to efficiently utilize all processors while maintaining performance, load balance, and fairness. Multiprocessor scheduling is more complex than single-CPU scheduling because the OS must decide which process runs on which processor and how to balance load across CPUs.

Features of Multiprocessor Scheduling:

1. Load Sharing / Load Balancing:

- Tasks are distributed across multiple processors.

- Prevents one CPU from being overloaded while others remain idle.
- Improves overall system throughput.

2. Symmetric Multiprocessing (SMP):

- All processors share the same OS and memory.
- Each processor is identical and can execute any task.
- Scheduling decisions are shared among CPUs.

3. Asymmetric Multiprocessing (AMP):

- One processor acts as master, others as slaves.
- Simple but less flexible.

4. Processor Affinity:

- Improves cache performance and reduces overhead.
- A process tends to run on the same processor it previously used.

5. Scalability:

- More processors -> higher throughput.
- Scheduling adapts as the number of CPUs increases.